Prompt: Write a recursive, object-oriented C++ program that solves a Sudoku puzzle on a 9x9 grid subdivided into 9 3x3 subgrids. Following that, if given a partially complete puzzle, it can find a possible solution if one exists or prove that no solution exists. Outputs expected are: puzzle file name, puzzle from the file printed out, solved puzzle printed out, or "No solution found" if it is proven not to be solvable through exhaustive means; if not unique solution, print all possible solutions. When creating solution.txt files, they should follow the naming convention: solution1.txt is tied to puzzle1.txt

GenAI used: I used 365 Copilot through from their desktop app and Claude from the web browser

Justification: I chose to use the claude generated code this time due to it using a bitmasking which I wanted to work more with since I have done that less I used both stack overflow and geeks for geeks to get more understanding of how to work with this as I went through and looked for what could be improved in the files. The time and space complexity were better with the claude version also making it a better choice to start with outside of my personal reason. The long term maintainability of the claude code was also generally more maintainable since it was decomposed into more classes that make it easier in the long run. I made a small improvement to the maintainability in adding the SolutionListenr and SolutionPrinter classes that will write out the outputs.

# Code generated by Claude (web browser)

```
// sudoku.cpp
// Recursive, object-oriented Sudoku solver (9x9 grid, nine 3x3 subgrids).
//
// For every puzzle file given, the program prints:
//   * the puzzle file name
//   * the puzzle read from the file
//   * every solution found (exhaustive search), or "No solution found"
// and writes the same solution output to a solution file named after the
// puzzle file: puzzle1.txt -> solution1.txt, puzzle2.txt -> solution2.txt, ...
//
// Build:  g++ -std=c++11 -O2 -Wall -Wextra -o sudoku sudoku.cpp
// Run:    ./sudoku puzzle1.txt [puzzle2.txt ...] [--limit N]
//         ./sudoku               (prompts for a puzzle file name)
//
// Puzzle file format: 81 cells, row by row. Digits 1-9 are given clues;
// 0 . _ * x X mark an empty cell. Whitespace and the characters | + - are
// ignored, so grids may also be drawn with box dividers. Windows (CRLF) line
// endings are fine.

#include <bitset>
#include <cctype>
#include <cstdlib>
#include <fstream>
#include <functional>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

// ---------------------------------------------------------------------------
// Board: owns the grid and answers "is this placement legal?" in O(1) using
// one bitmask per row, column and 3x3 subgrid.
// ---------------------------------------------------------------------------
class Board {
public:
    static const int SIZE = 9;
    static const int BOX = 3;

    Board() { clear(); }

    void clear() {
        for (int r = 0; r < SIZE; ++r) {
            rowMask_[r] = colMask_[r] = boxMask_[r] = 0;
            for (int c = 0; c < SIZE; ++c) cells_[r][c] = 0;
        }
        valid_ = true;
    }

    // Reads a puzzle from a file. Returns false (and sets err) on I/O or
    // format problems. If the clues themselves contradict each other the
    // board loads fine but isValid() reports false.
    bool loadFromFile(const std::string& path, std::string& err) {
        std::ifstream in(path.c_str());
        if (!in) {
            err = "could not open file";
            return false;
        }
        clear();
        std::vector<int> values;
        std::string line;
        while (std::getline(in, line)) {
            if (isSeparatorLine(line)) continue;
            for (std::size_t i = 0; i < line.size(); ++i) {
                char ch = line[i];
                if (ch >= '1' && ch <= '9')
                    values.push_back(ch - '0');
                else if (ch == '0' || ch == '.' || ch == '_' || ch == '*' ||
                         ch == 'x' || ch == 'X')
                    values.push_back(0);
                // anything else (spaces, |, +, -, commas...) is ignored
            }
        }
        if (values.size() != static_cast<std::size_t>(SIZE * SIZE)) {
            std::ostringstream os;
            os << "expected 81 cells but found " << values.size();
            err = os.str();
            return false;
        }
        for (int i = 0; i < SIZE * SIZE; ++i) {
            int v = values[i], r = i / SIZE, c = i % SIZE;
            if (v == 0) continue;
            if (canPlace(r, c, v))
```

```
                place(r, c, v);
            else
                valid_ = false;  // two clues conflict -> unsolvable
        }
        return true;
    }

    bool isValid() const { return valid_; }
    int get(int r, int c) const { return cells_[r][c]; }
    bool isEmpty(int r, int c) const { return cells_[r][c] == 0; }

    bool canPlace(int r, int c, int v) const {
        return (usedMask(r, c) & (1u << v)) == 0;
    }

    void place(int r, int c, int v) {
        cells_[r][c] = v;
        unsigned bit = 1u << v;
        rowMask_[r] |= bit;
        colMask_[c] |= bit;
        boxMask_[boxIndex(r, c)] |= bit;
    }

    void remove(int r, int c) {
        unsigned bit = 1u << cells_[r][c];
        cells_[r][c] = 0;
        rowMask_[r] &= ~bit;
        colMask_[c] &= ~bit;
        boxMask_[boxIndex(r, c)] &= ~bit;
    }

    // Bitmask of digits (bits 1..9) that may legally go in (r, c).
    unsigned candidates(int r, int c) const {
        return ~usedMask(r, c) & 0x3FEu;
    }

    static int countBits(unsigned m) { return static_cast<int>(std::bitset<16>(m).count()); }

    // Plain grid, one row per line, each cell followed by a space (the same
    // layout as the puzzle/solution files). Empty cells print as `blank`.
    std::string toString(char blank = '_') const {
        std::ostringstream os;
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c)
                os << (cells_[r][c] ? static_cast<char>('0' + cells_[r][c]) : blank) << ' ';
            os << '\n';
        }
        return os.str();
    }

private:
    int cells_[SIZE][SIZE];
    unsigned rowMask_[SIZE], colMask_[SIZE], boxMask_[SIZE];
    bool valid_;

    static int boxIndex(int r, int c) { return (r / BOX) * BOX + (c / BOX); }

    unsigned usedMask(int r, int c) const {
        return rowMask_[r] | colMask_[c] | boxMask_[boxIndex(r, c)];
    }

    // A line made only of - + = | and whitespace is a grid divider, not data.
    static bool isSeparatorLine(const std::string& s) {
        bool sawDivider = false;
        for (std::size_t i = 0; i < s.size(); ++i) {
            char ch = s[i];
            if (ch == '-' || ch == '+' || ch == '=' || ch == '|')
                sawDivider = true;
            else if (!std::isspace(static_cast<unsigned char>(ch)))
                return false;
        }
        return sawDivider;
    }
};

// ---------------------------------------------------------------------------
// Solver: recursive backtracking search that explores the ENTIRE search tree,
// so it finds every solution (or proves there is none). At each level it
// branches on the empty cell with the fewest candidates, which prunes hard.
// ---------------------------------------------------------------------------
class Solver {
public:
    typedef std::function<void(const Board&, unsigned long long)> SolutionCallback;

    // limit == 0 means "no limit": enumerate every solution.
    Solver(Board& board, unsigned long long limit = 0)
```

```
        : board_(board), limit_(limit), count_(0), limitHit_(false) {}

    // Runs the search; onSolution is called once per solution as it is found.
    // Returns the number of solutions found.
    unsigned long long solveAll(const SolutionCallback& onSolution) {
        count_ = 0;
        limitHit_ = false;
        callback_ = onSolution;
        if (board_.isValid()) search();
        return count_;
    }

    bool hitLimit() const { return limitHit_; }

private:
    Board& board_;
    unsigned long long limit_, count_;
    bool limitHit_;
    SolutionCallback callback_;

    // The recursive step.
    void search() {
        int bestR = -1, bestC = -1, bestCount = 10;
        unsigned bestMask = 0;

        for (int r = 0; r < Board::SIZE; ++r) {
            for (int c = 0; c < Board::SIZE; ++c) {
                if (!board_.isEmpty(r, c)) continue;
                unsigned m = board_.candidates(r, c);
                int n = Board::countBits(m);
                if (n < bestCount) {
                    bestCount = n;
                    bestR = r;
                    bestC = c;
                    bestMask = m;
                    if (n == 0) return;  // dead end: backtrack
                }
            }
        }

        // Base case: no empty cells left -> a complete, valid solution.
        if (bestR == -1) {
            ++count_;
            if (callback_) callback_(board_, count_);
            if (limit_ != 0 && count_ >= limit_) limitHit_ = true;
            return;
        }

        // Recursive case: try each candidate digit, then undo it.
        for (int v = 1; v <= 9; ++v) {
            if (!(bestMask & (1u << v))) continue;
            board_.place(bestR, bestC, v);
            search();
            board_.remove(bestR, bestC);
            if (limitHit_) return;
        }
    }
};

// ---------------------------------------------------------------------------
// Helpers for file names and per-puzzle processing
// ---------------------------------------------------------------------------

// puzzle1.txt -> solution1.txt (same directory). Names without "puzzle" in
// them become solution_<name>.
static std::string solutionFileName(const std::string& puzzlePath) {
    std::size_t slash = puzzlePath.find_last_of("/\\");
    std::string dir = (slash == std::string::npos) ? "" : puzzlePath.substr(0, slash + 1);
    std::string base = (slash == std::string::npos) ? puzzlePath : puzzlePath.substr(slash + 1);

    std::string lower = base;
    for (std::size_t i = 0; i < lower.size(); ++i)
        lower[i] = static_cast<char>(std::tolower(static_cast<unsigned char>(lower[i])));

    std::size_t pos = lower.find("puzzle");
    if (pos != std::string::npos)
        base.replace(pos, 6, "solution");
    else
        base = "solution_" + base;
    return dir + base;
}

static void processPuzzle(const std::string& path, unsigned long long limit) {
    std::cout << "================================================\n";
    std::cout << "Puzzle file: " << path << "\n\n";
    Board board;
```

```
    std::string err;
    if (!board.loadFromFile(path, err)) {
        std::cout << "Error: " << err << "\n\n";
        return;
    }

    std::cout << "Puzzle:\n" << board.toString() << '\n';

    std::string outName = solutionFileName(path);
    std::ofstream out(outName.c_str());
    if (!out) std::cout << "Warning: could not create " << outName << "\n\n";

    // Each solution is a grid followed by a blank line; the file/console then
    // ends with one more blank line and "Solutions found: N".
    Solver solver(board, limit);
    unsigned long long count = solver.solveAll(
        [&](const Board& b, unsigned long long n) {
            if (n == 1) std::cout << "Solution:\n";
            std::cout << b.toString() << '\n';
            if (out) out << b.toString() << '\n';
        });

    if (count == 0) {
        std::cout << "No solution found\n\n";
        if (out) out << "No solution found\n";
    } else {
        std::ostringstream summary;
        summary << "Solutions found: " << count
                << (solver.hitLimit() ? " (stopped at --limit)" : "");
        std::cout << '\n' << summary.str() << "\n\n";
        if (out) out << '\n' << summary.str() << '\n';
    }
    if (out) std::cout << "Output written to " << outName << "\n\n";
}

int main(int argc, char* argv[]) {
    std::vector<std::string> files;
    unsigned long long limit = 0;  // 0 = exhaustive, print every solution

    for (int i = 1; i < argc; ++i) {
        std::string a = argv[i];
        if (a == "--limit" && i + 1 < argc)
            limit = std::strtoull(argv[++i], NULL, 10);
        else
            files.push_back(a);
    }

    if (files.empty()) {
        std::string name;
        std::cout << "Enter puzzle file name: ";
        if (!std::getline(std::cin, name) || name.empty()) {
            std::cerr << "No puzzle file given.\n";
            return 1;
        }
        files.push_back(name);
    }

    for (std::size_t i = 0; i < files.size(); ++i) processPuzzle(files[i], limit);
    return 0;
}
```

# Code generated by Microsoft 365 Copilot (desktop app)

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

class Sudoku
{
private:
    int board[9][9];
    vector<vector<vector<int>>> solutions;
    string puzzleFileName;

    bool isValid(int row, int col, int num)
    {
        // Check row
        for (int c = 0; c < 9; c++)
        {
            if (board[row][c] == num)
                return false;
        }

        // Check column
        for (int r = 0; r < 9; r++)
        {
            if (board[r][col] == num)
                return false;
        }

        // Check 3x3 subgrid
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        for (int r = startRow; r < startRow + 3; r++)
        {
            for (int c = startCol; c < startCol + 3; c++)
            {
                if (board[r][c] == num)
                    return false;
            }
        }

        return true;
    }

    bool findEmpty(int &row, int &col)
    {
        for (row = 0; row < 9; row++)
        {
            for (col = 0; col < 9; col++)
            {
                if (board[row][col] == 0)
                    return true;
            }
        }
        return false;
    }

    void storeSolution()
    {
        vector<vector<int>> solution(9, vector<int>(9));

        for (int r = 0; r < 9; r++)
        {
            for (int c = 0; c < 9; c++)
            {
                solution[r][c] = board[r][c];
            }
        }

        solutions.push_back(solution);
    }

    void solveRecursive()
    {
        int row, col;

        if (!findEmpty(row, col))
        {
            storeSolution();
            return;
```

```
        }

        for (int num = 1; num <= 9; num++)
        {
            if (isValid(row, col, num))
            {
                board[row][col] = num;

                solveRecursive();

                // Backtrack
                board[row][col] = 0;
            }
        }
    }

public:
    Sudoku(const string &fileName)
    {
        puzzleFileName = fileName;

        ifstream infile(fileName);

        if (!infile)
        {
            cerr << "Error opening file: "
                 << fileName << endl;
            exit(1);
        }

        for (int r = 0; r < 9; r++)
        {
            for (int c = 0; c < 9; c++)
            {
                infile >> board[r][c];
            }
        }

        infile.close();
    }

    void printPuzzle()
    {
        for (int r = 0; r < 9; r++)
        {
            for (int c = 0; c < 9; c++)
            {
                cout << board[r][c] << " ";
            }
            cout << endl;
        }
    }

    void solve()
    {
        solveRecursive();
    }

    void printSolutions()
    {
        if (solutions.empty())
        {
            cout << "\nNo solution found.\n";
            return;
        }

        cout << "\nTotal Solutions Found: "
             << solutions.size()
             << endl;

        for (size_t s = 0; s < solutions.size(); s++)
        {
            cout << "\nSolution "
                 << s + 1
                 << ":\n";

            for (int r = 0; r < 9; r++)
            {
                for (int c = 0; c < 9; c++)
                {
                    cout << solutions[s][r][c] << " ";
                }
                cout << endl;
            }
        }
    }
```

```
    void writeSolutionFile()
    {
        if (solutions.empty())
        {
            return;
        }

        string solutionFile = puzzleFileName;

        size_t pos = solutionFile.find("puzzle");

        if (pos != string::npos)
        {
            solutionFile.replace(pos, 6, "solution");
        }

        ofstream outfile(solutionFile);

        if (!outfile)
        {
            cerr << "Could not create "
                 << solutionFile << endl;
            return;
        }

        for (int r = 0; r < 9; r++)
        {
            for (int c = 0; c < 9; c++)
            {
                outfile << solutions[0][r][c] << " ";
            }
            outfile << endl;
        }

        outfile.close();

        cout << "\nSolution saved to "
             << solutionFile
             << endl;
    }
};

int main()
{
    string fileName;

    cout << "Enter puzzle file name: ";
    cin >> fileName;

    Sudoku puzzle(fileName);

    cout << "\nPuzzle File: "
         << fileName
         << endl;

    cout << "\nOriginal Puzzle:\n";
    puzzle.printPuzzle();

    puzzle.solve();

    cout << "\nSolved Puzzle(s):\n";
    puzzle.printSolutions();

    puzzle.writeSolutionFile();

    return 0;
}
```